

SENTINEL АНТИ-РУКОВОДСТВО

Том I: Плоская память и другие формы эмоционального ущерба

Этот документ не требуется для изучения Sentinel.

Он требуется для того, чтобы понять, почему автора нельзя оставлять наедине с Markdown.

ВНИМАНИЕ

Данный документ может содержать:

недокументированную мудрость

задокументированную чушь

избыточное количество архитектурной «воды»

прерывания на соленые огурцы

философию жареных пельменей

психологический ущерб

ровно одно полезное предложение на главу

попытки переплюсовать C++

Если вы пришли сюда, чтобы быстро выучить Sentinel — уходите. Читайте Гайд.

Если вы пришли сюда, чтобы красиво страдать —
продолжайте.

ГЛАВА 0 — Перед ключевым словом

До того как появился `local`, не было ничего.

Не было даже памяти.

Не было даже надежды.

Только сырой бинарный ветер, гуляющий по пустому
загрузочному сектору.

Затем программист ввел:

```
local counter = 0
```

И плоская вселенная открыла свои глаза.

Машина не улыбнулась. Машины редко улыбаются.

Но где-то внутри NASM одна метка приготовилась.

```
sl_var_counter dq 0
```

Это была не переменная.

Это было объявление намерений. Обещание.

Ячейка памяти, выкованная в подвале причинно-следственных связей.

Читатель может спросить:

«Почему мы тратим столько строк на local?»

Ответ прост. Мы не объясняем. Мы увеличиваем количество страниц.

ПРИМЕЧАНИЕ 0.1 — О «воде»

Вода, которая сейчас льется в это руководство — непростая. Это архитектурная жидкость документационного класса.

Она ничего не улучшает. Она мало что проясняет.

Она просто делает главу солидной на вид.

Именно так и рождаются стандарты.

ПРИМЕЧАНИЕ 0.2 — Ещё больше «воды»

Если читатель считает, что предыдущее примечание было лишним, значит, читатель верно определил цель этого руководства.

Вода здесь — не баг. Вода — это фича.

Вода — это среда выполнения (runtime) корпоративной документации.

ГЛАВА 1 — local

local объявляет плоскую память.

Это было то самое полезное предложение. Теперь вы можете закрыть главу.

Однако, поскольку это Анти-Руководство, мы продолжаем.

local — это не предложение. local — это не вайб. local — это не крошечная переменная, прячущаяся внутри уютной области видимости (scope).

Здесь нет уютных областей видимости. Нет укромных местечек.

Есть только плоская плоскость.

Плоская плоскость видит все имена. Плоская плоскость помнит все объявления. Плоской плоскости плевать, что вы «просто хотели временную переменную на секунду».

Если вы напишете:

```
local a = 10
```

```
local a = 20
```

Компилятор отвергнет вас. Не ваш код. Вас.

[SEMANTIC S003]

А программа — это так, сопутствующий ущерб.

ПРИМЕЧАНИЕ 1.1 — Двойное именование

Назвать вещь один раз — это architecture.

Назвать её дважды — это место преступления.

Sentinel не спрашивает «почему». Sentinel не собирается посещать твою психотерапию по неймингу. Sentinel выплевывает S003 и идет дальше.

ГЛАВА 2 — Почему области видимости вас не спасут

Классические программисты обожают области видимости (scopes). Они строят маленькие комнатки для переменных. Они закрывают двери. Они шепчут: «Это чистая архитектура».

Sentinel смотрит на это и говорит:

«Мило».

В Sentinel v0.3-alpha память плоская. И это не потому, что автор забыл про существование областей видимости. Это потому, что загрузочному коду плевать на вашу эмоциональную мебель.

У бутлоадера есть 512 байт. Ему не нужен ваш абстрактный дворец. Ему нужна определенность.

```
local boot_stage = 0
```

Вот и всё. Стадия существует. Все её видят. И никто не делает вид, будто она спрятана за тремя блоками кода и лайфстайл-фреймворком.

ПРИМЕЧАНИЕ 2.1 — О лайфстайл-фреймворках

Лайфстайл-фреймворк — это то, что происходит, когда переменная хочет себе дом, контейнер внедрения зависимостей (DI), три фабрики и доклад на конференции.

Sentinel предлагает память. Бери её или возвращайся в свой собор.

ГЛАВА 3 — redo

redo мутирует существующую память.

Это было полезное предложение.

Но автору скучно. Поэтому мы продолжаем.

Вы можете подумать, что redo означает присваивание. Это правда лишь отчасти. redo — это присваивание с памятью о предательстве.

Когда вы пишете:

```
redo: counter to counter + 1
```

Вы не создаете counter. Вы возвращаетесь к нему. Вы трогаете то, что уже существует. Вы говорите:

«Я помню тебя, ячейка памяти».

Если памяти не существует, Sentinel отказывает.

[SEMANTIC S008]

Перевод: «Ты не можешь изменить призрака, гений».

ПРИМЕЧАНИЕ 3.1 — О мутации призраков

Некоторые языки позволяют хаосу становиться поведением в рантайме.

Sentinel prefers прекращать этот спиритический сеанс на ранней стадии.

Если ты хочешь мутировать имя — сначала вызови его к жизни.

Это не жестокость. Это базовая метафизика.

ГЛАВА 4 — Секрет redo

В этот момент автор раскрывает секрет о redo. Этот секрет не скрыт ни в одном публичном документе. Ладно, это ложь. Он есть в спецификации. Но драматическое напряжение улучшает вкус документации.

Секрет в следующем:

redo это redo.

Читатель может взять паузу, чтобы прийти в себя.

УПРАЖНЕНИЕ 4.1

Объясните redo.

Ожидаемый ответ: redo.

Альтернативный принятый ответ: мутация, но с эмоциональными последствиями.

ПРИМЕЧАНИЕ 4.2 — Дополнительное объяснение

Если предыдущего объяснения было недостаточно, пожалуйста, перечитывайте строку 200 до тех пор, пока не наступит просветление или обезвоживание.

ГЛАВА 5 — Шаги функций

Функции в Sentinel имеют пронумерованные шаги.

`create boot()`

(1) `console_print("cpu")`

(2) `console_print("memory")`

(3) `console_print("drivers")`

Некоторые языки вызывают функции. Sentinel совершает церемонии.

Шаг первый. Шаг второй. Шаг третий.

Это не оверинжиниринг. Это грамотность последовательности загрузки.

Ядро не инициализируется как облако вайбов. Оно просыпается по стадиям.

Процессор. Память. Прерывания. Планировщик. Паника.

Обычно паника.

ПРИМЕЧАНИЕ 5.1 — О Панике

Паника — это не провал. Паника — это ядро, которое честно выражает свои чувства.

Молчаливое ядро хуже. У молчаливого ядра есть секреты.

ГЛАВА 6 — Пропущенные шаги

Если вы напишете:

```
start boot(9)
```

но у boot есть только шаги 1, 2 и 3, Sentinel не заставит NASM страдать.

Sentinel остановит вас раньше.

[SEMANTIC S012]

Перевод: «Бро, девятого шага не существует».

Это милосердие. Низкоуровневое милосердие, но всё же.

Си позволил бы вам построить ракету. Ракета покинула бы атмосфера. Десять минут спустя указатель вспомнил бы, что у него не было детства. И тогда все превращаются в телеметрию.

Sentinel prefers оскорбить вас до запуска.

ПРИМЕЧАНИЕ 6.1 — Ракетная инженерия

Если твоя ракета зависит от неопределенного символа, это не ракета.

Это судебный спор с гравитацией.

ГЛАВА 7 — Параметры

Параметры функций — это временные имена для входных данных. Они вам не память. Они вам не друзья. Это гости с привкусом регистров.

```
create add(left, right)
```

В x64 первый гость садится в rdi. Второй гость садится в rsi. Третий гость садится в rdx.

Они пришли на ужин. Не нужно перестраивать дом ради них.

Если вам нужен выходной результат — объявите память.

```
local add_out = 0
```

А затем мутируйте её.

```
redo: add_out to left + right
```

Это называется дисциплиной. Многие программисты её боятся.

ПРИМЕЧАНИЕ 7.1 — О гостях

Параметр — это гость. Ячейка памяти — это мебель. Если ты попытаешься сделать ремонт прямо на госте, компилятор начнет сильно беспокоиться.

ГЛАВА 8 — Инцидент с RSI

Когда-то давно `console_print` использовал регистр `rsi`. Это было нормально. Строкам нужны указатели. Указателям нужны регистры. Регистрам нужна эмоциональная поддержка.

К несчастью, `rsi` также был вторым аргументом функции.

И произошло следующее:

```
create add(x, y)
```

```
(1) console_print("before add")
```

```
(2) redo: out to x + y
```

Печать украла `y`. Функция сложила `x` с адресом строки. Машина не пожаловалась. Машина просто молча соучаствовала в преступлении.

Версия v0.3-alpha исправила это.

```
push rsi
```

```
call sl_print_str
```

```
pop rsi
```

Это не просто кодогенерация. Это развитие персонажа.

ПРИМЕЧАНИЕ 8.1 — Терапия для регистров

Каждый регистр заслуживает личных границ. Особенно rsi.

Он видел строки. Он видел аргументы. Ему нужен отдых.

ГЛАВА 9 — Вызов шагов функций с параметрами

Не делайте так:

```
start add(2)
```

если у add есть параметры.

Вы думаете, что вызываете шаг 2. На самом деле вы телепортируетесь в функцию, забыв взять с собой аргументы.

Это не оптимизация. Это селф-харм с помощью меток.

[SEMANTIC S020]

Перевод: «Ты вошел в комнату сквозь стену и забыл штаны».

ПРИМЕЧАНИЕ 9.1 — Телепортация

Телепортация безопасна только тогда, когда ты твердо знаешь, где находятся твои регистры.

Компилятор штаны не выдает.

ГЛАВА 10 — Числовые литералы в `start()`

В вызове `start func(1)` единица — это селектор шага. Это не данные.

Если вам нужны данные — объявите их.

```
local amount = 1
```


start func(amount)

Оно существует потому, что двусмысленность — это то, как языки становятся проклятыми.

Sentinel и так низкоуровневый. Дополнительные демоны ему не нужны.

ПРИМЕЧАНИЕ 10.1 — Бюджет на демонов

У каждого компилятора есть бюджет на демонов. Sentinel тратит большую его часть на генерацию вывода под NASM.

На неоднозначные литералы бюджета просто не осталось.

ГЛАВА 11 — console_print

console_print выводит текст.

Это было полезное предложение.

А теперь давайте обсудим, как вывод одного символа в низкоуровневом мире требует продажи души.

Человек высокоуровневый:

```
print("A")
```

Человек низкоуровневый:

Найти видеопамять.

Проверить режим.

Помолиться сегментным регистрам.

Записать байт.

Записать атрибут.

Промахнуться мимо адреса.

Черный экран.

Переосмыслить карьеру.

Sentinel говорит:

```
console_print("A")
```

Но глубоко внутри демон всё ещё пишет в VGA. Мы не изгнали демона. Мы просто дали ему интерфейс посимпатичнее.

ПРИМЕЧАНИЕ 11.1 — О VGA

Текстовый режим VGA древний. Древние вещи либо сломаны, либо священные.

VGA — и то, и другое одновременно.

ГЛАВА 12 — low-code

low-code — это место, где Sentinel перестает притворяться.

low-code:

```
emit 0x90
```

Это NOP. Он ничего не делает. Что делает его одной из самых безопасных инструкций, когда-либо изобретенных человечеством.

Многие программы стали бы лучше, если бы их заменили на NOP'ы. Это не шутка. Это инженерия производительности.

low-code позволяет вам извергать байты. Используйте его осторожно.

Если вы не знаете, что делает байт, байт знает, что сделали вы.

ПРИМЕЧАНИЕ 12.1 — О честности байтов

Байты не лгут. Лгут программисты.

Машина помнит байт. Отладчик помнит твой позор.

ГЛАВА 13 — FREERAM

`FREERAM(x)` на данный момент просто очищает память.

Она не призывает профессиональный аллокатор памяти из пустоты. Она не исправит вашу жизнь. Она не оплатит ваш VPS.

Она просто очищает память.

Это полезно. Но это не магия.

Если вы вызовете:

`FREERAM(missing)`

Sentinel скажет:

[SEMANTIC S019]

Перевод: «Очистить что, прости?»

ПРИМЕЧАНИЕ 13.1 — О свободе

Чтобы освободить память, память для начала должна существовать.

Это скандальное заявление чудом прошло рецензирование здравым смыслом.

ГЛАВА 14 — Жареные пельмени

Жареные пельмени превосходны.

...

Впрочем, как и 37% документов по корпоративной архитектуре, за написание которых людям всё равно платят деньги.

Программисты потребляют. Программисты компилируют. Программисты наблюдают за вселенной сквозь масло, соль и неразрешенные ошибки линкера.

В этот момент автор прерывается, чтобы поест
жареных пельменей.

...

Мы продолжаем.

ПРИМЕЧАНИЕ 14.1 — Питательная архитектура

Жареные пельмени дают калории. Калории дают
скорость печати. Скорость печати дает баги. Баги дают
документацию.

Следовательно, жареные пельмени являются
официальной частью тулчейна разработки.

ГЛАВА 15 — Прерывание на соленый огурец

`std` — это оружие из железобетона.

Погодите.

Автор ест огурец с солью.

...

Мы снова продолжаем.

Некоторые оскорбленные фанаты C/C++ могут назвать `std` мусором. Их мнение валидно — примерно так же, как загрузчик без сигнатуры `0xAA55` технически является файлом.

Он существует. Он просто фейлит единственную вещь, ради которой был рожден.

ПРИМЕЧАНИЕ 15.1 — О соли

Лапша быстрого приготовления не станет лучше, если ты бахнешь туда соль уже после пакетика со специями.

Это не имеет прямого отношения к Sentinel Lang.

Впрочем, как и многие дизайн-документы, в которых зачем-то всё равно рисуют диаграммы.

ГЛАВА 16 — `lib(std)`

`lib(std)` планируется. Её ещё здесь нет. Успокойтесь.

Цель `std` — не стать собором. Цель — предоставить острые инструменты для OSDev.

```
halt()
```

```
panic("bad")
```

```
read_port(0x60)
```

```
write_port(0x20, 0x20)
```

```
vga_print("ok")
```

Это не роскошь. Это выживание.

Си говорит:

```
__asm__ volatile ("inb %1, %0" : "=a"(ret) : "Nd"(port));
```

Sentinel хочет говорить:

```
read_port(0x60)
```

Это не слабость. Это цивилизация.

ПРИМЕЧАНИЕ 16.1 — О цивилизации

Цивилизация начинается тогда, когда портовый ввод-вывод (I/O) перестает выглядеть как юридический контракт.

Общество, способное написать `read_port(0x60)`, ещё можно спасти.

ГЛАВА 17 — С

Си — это нож. Хороший нож. Легендарный нож. Нож, с помощью которого создавались операционные системы, компиляторы, игры, драйверы и кошмары.

Но Си позволит вам ударить себя ножом и назовет это неопределенным поведением (undefined behavior).

Sentinel не утверждает, что он везде сильнее Си. Sentinel утверждает, что, возможно, совсем чуть-чуть, загрузочный код мог бы перестать выглядеть как записка с требованием выкупа, написанная линкером.

ПРИМЕЧАНИЕ 17.1 — Неопределенное поведение

Неопределенное поведение (UB) — это когда язык смотрит на реальность и заявляет: «Это не мой отдел».

Программа может сработать. Программа может взорваться. Программа может подождать сорок минут и только потом испортить вам вечер.

ГЛАВА 18 — C++

C++ — это космическая станция, сделанная из ножей.

Некоторые комнаты прекрасны. Некоторые комнаты содержат шаблоны. Некоторые комнаты содержат шаблоны, генерирующие шаблоны.

Без библиотек C++ всё еще силен. Но он тяжелый.

Sentinel не пытается пере-сиплюснуть C++. Sentinel пытается зайти в OSDev с ломом и читаемым синтаксисом.

Это другой спорт.

ГЛАВА 18.1 — Переплюснуть C++

Многие языки пытались победить C++.

Немногие осмелились его переплюснуть.

Переплюснуть C++ — это не значит стать больше. Это значит стать острее.

У C++ два плюса.

У Sentinel — ни одного.

Именно поэтому Sentinel движется быстрее.

У него меньше аэродинамическое сопротивление знаков препинания.

Программист C++ может возразить:

«Но как же шаблоны, constexpr, RAll, метапрограммирование, концепты...»

Программист Sentinel отвечает:

local patience = 0

redo: patience to patience - 1

Это не контраргумент. Это выбор стиля жизни.

ПРИМЕЧАНИЕ 18.1.1 — Формальное определение

Переплюсовать C++: низкоуровневый лингвистический маневр, при котором меньший язык пытается одолеть крупную сущность на базе плюсов, наотрез отказываясь превращаться в гидру.

Сопутствующие термины:

шаблонное погодное явление

туман абстракции

гроза времени компиляции

болото перегрузки операторов

ПРИМЕЧАНИЕ 18.1.2 — О втором плюсе

Первый плюс добавил мощи. Второй плюс добавил ответственности.

Третий плюс так и не был добавлен. Человечество выжило.

Sentinel содержит ровно ноль плюсов в своем названии. Это оптимизация.

ГЛАВА 19 — NASM

NASM — не ваш враг. NASM — это зеркало.

Если NASM кричит, значит, что-то долетело до него слишком поздно.

Версия v0.3-alpha пытается это предотвратить.

Сломанный Sentinel должен падать как Sentinel.

А не как древний раскат ассемблерного грома.

ПРИМЕЧАНИЕ 19.1 — Ассемблерный гром

Ассемблерный гром — это звук, возникающий, когда компилятор отправляет откровенную чушь в храм и ожидает, что храм проявит вежливость.

NASM редко бывает вежливым. NASM точен.

Точность кажется грубостью, когда ты не прав.

ГЛАВА 20 — Семантические ошибки

Семантические ошибки — это не провалы. Они — вмешательство компилятора ради вашей безопасности.

[SEMANTIC S012]

означает: «У функции нет такого шага».

Но эмоционально это означает: «Я остановил тебя до того, как твой ассемблер стал археологической находкой».

[SEMANTIC S020]

означает: «Не вызывайте шаги функций, требующих параметров».

Но духовно это означает: «Не телепортируйся без регистров».

ПРИМЕЧАНИЕ 20.1 — О спасении

Пользователь часто принимает семантическую ошибку за проявление враждебности. Это неверно.

Компилятор не злится. Компилятор разочарован на ранней стадии.

Раннее разочарование обходится гораздо дешевле поздней отладки.

ГЛАВА 21 — Гайд

Гайд учит Sentinel. Это руководство — нет.

Гайд говорит:

local x = 0

объявляет память.

Это руководство говорит:

local x = 0

— это миг, когда ячейка памяти пробуждается под холодной луной загрузочного пространства.

Оба утверждения технически совместимы. Просто одно тратит больше бумаги.

Именно поэтому должны существовать оба документа.

ПРИМЕЧАНИЕ 21.1 — Об образовании

Хороший гайд помогает читателю. Хорошее анти-руководство проверяет, осталась ли в читателе хоть какая-то радость.

ГЛАВА 22 — Сайт

Однажды у Sentinel появится свой сайт. На нем будут:

Гайд. Спецификация. Примеры. Дорожная карта (Roadmap). Семантические ошибки. Анти-Руководство.

Анти-Руководство станет самой посещаемой страницей.

Это вызовет беспокойство у серьезных людей. Серьезные люди спросят: «Почему шуточная документация популярнее?»

Ответ будет: «Потому что она содержит по одному полезному предложению на главу».

ГЛАВА 23 — Последнее предупреждение

Sentinel не закончен.

Sentinel не стабилен.

Sentinel здесь не для того, чтобы завтра заменить каждый язык на планете.

Sentinel — это маленькое низкоуровневое существо с опасным потенциалом.

У него есть `local`.

У него есть `redo`.

У него есть пошаговые функции.

У него есть семантические ошибки.

У него есть воля к тому, чтобы переплюсовать С++.

И этого уже достаточно для беспокойства.

ПРИМЕЧАНИЕ 23.1 — О беспокойстве

Беспокойство — это то, что происходит, когда игрушечный компилятор внезапно начинает генерировать загрузочные сектора, x64-бинарники, семантическую диагностику и шутки, которые случайно объясняют устройство языка.

ВАЖНОЕ ПРИМЕЧАНИЕ 23.1.1 — Преданность компилятора и лапша быстрого приготовления

И да, я налью сюда ещё немного воды.

Запомните следующее:

Компилятор ненавидит вас всех.

Почему? Всё просто. Вы небрежны.

Вы заявляете сюда с продублированной памятью, пропущенными шагами, неизвестными символами, сломанными вызовами шагов и уверенностью людей, которые думают, что NASM — это психотерапевт.

Я создал этот компилятор. Он работает.

Что это значит?

Нет, это не значит, что я лучше вас.

Это значит, что лапша быстрого приготовления отвратительна даже без пакетика со специями.

Никогда не добавляйте лишнюю соль в быстрорастворимую лапшу.

Это утверждение не имеет прямого отношения к Sentinel.

Впрочем, как и половина этой главы, но вы же всё равно продолжили читать.

Вернемся к компилятору.

Компилятор ненавидит вас не потому, что он злой.

Он ненавидит вас потому, что увидел ваш код раньше, чем вы сами.

Он знает, что вы пытались сделать.

Он знает, почему это было ошибкой.

И он знает, что вы собирались обвинить во всем компилятор.

Возможные варианты действий:

Перестать быть небрежным.

Объявлять память перед её использованием.

Не вызывать шаги, которых не существует.

Не добавлять соль в лапшу быстрого приготовления.

Если компилятор отверг вас, задумайтесь о личностном росте.

ГЛАВА 24 — Последнее упражнение

Объявите память.

local sanity = 1

Мутируйте её.

redo: sanity to sanity - 1

Наблюдайте за результатом.

sanity == 0

Поздравляем. Вы прошли Анти-Руководство.

Возможные варианты действий:

Прочитать настоящий Гайд.

Потрогать траву.

Съесть огурец с солью.

Не переплюсплюшивать C++ до завтрака.

КОНЕЦ ПЕРВОГО ТОМА

Компилятор не отверг ваш код.

Компилятор отверг ваши оправдания.

Пожалуйста, переходите к Тому II:

«ABI, Сожаление и Регистр, который вас предал».